LEXAI

AI-Powered Legal Assistant & Contract Intelligence Platform

Project Handover & Technical Documentation

Version: 1.0

Status: 86/86 Tests Passing

Target Audience: MCA Final-Year Project Teammates

Table of Contents

1. Executive Project Overview
2. Problem Statement
3. Project Objectives
4. Complete Feature List
5. Technology Stack
6. Technology Concepts for Teammates
7. System Architecture
8. Complete User Flow
9. Public Website vs Application
10. Folder Structure
11. File-By-File Responsibility
12. Database Architecture
13. Document Processing Pipeline
14. AI Legal Analysis Pipeline
15. RAG System Explained
16. Knowledge Base
17. Chat System
18. Authentication & Security
19. Error Handling
20. Testing
21. Project Development Phases
22. Current Project Status
23. Gemini API / Quota
24. Environment Variables
25. How to Setup Project Locally
26. How to Run the Project

27. How to Add a New Knowledge Base Article

28. How to Debug Common Problems

29. Development Rules for Teammates

30. Project Limitations

31. Future Scope

32. Team Handover Section

33. Glossary

1. Executive Project Overview

What is LexAI?
LexAI is an AI-assisted legal information and document intelligence platform. It allows users to upload complex legal contracts (PDF or DOCX), automatically extracts the text, and leverages Google's Gemini Large Language Model (LLM) to perform deep legal analysis, producing structured executive summaries and clause breakdowns.

What problem does it solve?
Legal documents are notorious for being dense, lengthy, and filled with difficult jargon. Laypeople and legal professionals alike spend excessive time reviewing standard contracts to find obligations, liabilities, and core terms. Furthermore, accessing reliable general legal information often requires expensive consultations. LexAI automates the heavy lifting of document review and provides an instant, AI-driven knowledge base.

Who is it designed for?
LexAI is designed for individuals reviewing personal contracts, small business owners examining vendor agreements, and law students or professionals who need a fast, AI-powered paralegal assistant to highlight key elements of a legal text.

What can a user do with it?
Users can create an account, upload legal documents, view automated AI analyses of those documents, and interact with an AI Chatbot in two distinct modes: 1. **Document Chat:** Ask specific questions about an uploaded contract (e.g., "What is the termination clause?"). 2. **General Legal Chat:** Ask broad legal questions which the AI answers by retrieving grounded information from the platform's trusted Knowledge Base.

Why is AI useful in this project?
Traditional software can search for keywords, but only modern LLMs can understand the semantic meaning of a "force majeure" clause even if it's worded differently. AI enables the system to summarize, interpret, and answer natural language questions about dense text.

> **Disclaimer (As implemented in the project):**
> LexAI provides AI-generated information for educational and informational purposes only. It is **NOT** a replacement for a qualified lawyer and does not constitute professional legal advice. Laws vary by jurisdiction and circumstances.

2. Problem Statement

Navigating the legal landscape presents several distinct challenges that LexAI aims to mitigate: - **Complexity and Length:** Legal documents (contracts, terms of service, NDAs) are lengthy and difficult for non-lawyers to understand. - **Hidden Risks:** Users struggle to identify important clauses, rights, obligations, and potential liabilities buried in "legalese." - **Time Consumption:** Traditional document review requires significant manual effort and hours of reading. - **Access to Information:** Finding relevant, structured, and easy-to-understand legal information is time-consuming and often requires consulting expensive professionals.

3. Project Objectives

The core objectives of the LexAI project are defined below.

Implemented Objectives: - [x] Secure user authentication (registration, login, logout). - [x] Upload and parse legal documents (PDF and DOCX formats). - [x] Generate automated AI Legal Analyses (Executive Summary, Clause Breakdown). - [x] Provide a Retrieval-Augmented Generation (RAG) backend for natural language queries. - [x] Provide Document-based Chat (chatting with a specific uploaded file). - [x] Provide General Legal Chat grounded in a trusted Knowledge Base. - [x] Maintain secure user-specific document access and isolation. - [x] Maintain an Administrator system for managing the Knowledge Base. - [x] Handle AI API failures gracefully with deterministic fallback responses.

Planned / Future Objectives: - [ ] Multilingual legal support (Hindi, Marathi, etc.). - [ ] OCR for scanned/image-based legal documents. - [ ] Persistent cloud storage deployment (Firebase/AWS).

4. Complete Feature List

Feature	Description	Implemented?	Main Files
Authentication	Registration, Login, Logout, password hashing	Yes	`routes/auth_routes.py`
Dashboard	Overview of uploaded files and recent activity	Yes	`routes/main_routes.py`
Document Upload	Secure handling of `.pdf` and `.docx` uploads	Yes	`routes/document_routes.py`
PDF/DOCX Extraction	Text extraction using PyMuPDF and python-docx	Yes	`services/document_service.py`
Document Statistics	Calculating word, character, and page counts	Yes	`services/document_service.py`
AI Legal Analysis	Generating Executive Summaries via Gemini	Yes	`routes/document_routes.py`, `ai_service.py`
General Legal Chat	Chatbot answering general queries via KB	Yes	`routes/chat_routes.py`
Document Chat	Chatbot answering queries based on user doc	Yes	`routes/chat_routes.py`

Feature	Description	Implemented?	Main Files
RAG System	Vector similarity search for context injection	Yes	`services/rag_service.py`
Embeddings	Text chunking and embedding generation	Yes	`services/embedding_service.py`
Conversations & Messages	Saving chat histories to the database	Yes	`models/database_models.py`
Admin Panel	Protected UI for administrators	Yes	`routes/admin_routes.py`
KB Management	Admin CRUD operations for Knowledge Base	Yes	`routes/admin_routes.py`
Markdown Rendering	Displaying AI output beautifully (with MathJax)	Yes	`static/js/chatbot.js`
DOMPurify	Sanitizing markdown to prevent XSS attacks	Yes	`templates/chatbot.html`
Gemini Fallback	Deterministic KB fallback when Gemini is 503/429	Yes	`services/rag_service.py`
Automated Testing	Comprehensive Pytest suite checking functionality	Yes	`tests/`

5. Technology Stack

The LexAI project utilizes a modern Python-based web architecture.

Frontend

- **HTML5 & CSS3:** Standard markup and styling.
- **JavaScript (Vanilla):** Handles asynchronous chat requests, markdown rendering, and dynamic UI interactions without the overhead of heavy frameworks.
- **Bootstrap 5:** Used for the responsive grid, pre-built UI components (cards, modals, sidebars), and consistent styling.
- **marked.js & DOMPurify:** Used to safely render Markdown responses (from Gemini) into HTML on the client side while preventing Cross-Site Scripting (XSS).
- **MathJax:** Used to render complex mathematical/logical formulas (like contract equations) output by Gemini in LaTeX format.

Backend

- **Python (3.x):** The core programming language powering the backend logic.
- **Flask:** A lightweight WSGI web application framework used to route HTTP requests, manage sessions, and render templates.
- **Flask-SQLAlchemy:** An Object-Relational Mapper (ORM) that bridges Python objects and the MySQL database.
- **Flask-Login:** Manages user sessions, login state, and role-based access control.
- **Flask-Bcrypt:** Hashes user passwords securely using the bcrypt algorithm.

Database

- **MySQL (via mysql-connector-python):** The primary relational database storing users, documents metadata, analyses, chat histories, and knowledge base articles.
- **ChromaDB:** A specialized, locally-hosted Vector Database used entirely for storing and retrieving high-dimensional text embeddings for the RAG system.

AI & Machine Learning

- **Google GenAI SDK (`google-genai`):** Communicates with Google's Gemini API to generate intelligent text analysis and chatbot responses.

- **SentenceTransformers (`all-MiniLM-L6-v2`):** A local NLP model used to convert text (document chunks and queries) into 384-dimensional vector embeddings for semantic similarity search.

Document Processing

- **PyMuPDF (`fitz`):** A high-performance library used to parse and extract raw text from PDF files.
- **python-docx:** Used to parse and extract text from Microsoft Word (`.docx`) files.

Testing

- **Pytest:** The testing framework used to run the 86-test suite, ensuring routing, authentication, security, and RAG isolation behave as expected.

6. Technology Concepts for Teammates

To successfully contribute to LexAI, teammates should understand these core concepts:

Flask

Flask is a web framework for Python. Instead of writing raw HTTP servers, Flask allows us to map URLs to Python functions using decorators (e.g., `@app.route('/dashboard')`). It handles incoming requests, manages cookies/sessions, and returns HTML templates.

MySQL & Relational Databases

A relational database stores data in interconnected tables. LexAI uses MySQL to store structured metadata (like usernames, passwords, file paths, and conversation records). It ensures data integrity through relationships (e.g., a Document belongs to a User).

SQLAlchemy (ORM)

Writing raw SQL queries (like `SELECT * FROM users`) can be tedious and prone to SQL injection. SQLAlchemy allows us to interact with the database using Python objects (e.g., `User.query.all()`), automatically translating Python code into safe SQL queries behind the scenes.

Google Gemini API

Gemini is the Large Language Model (LLM) serving as the "brain" of LexAI. When a user uploads a contract, the backend extracts the text and sends it to Gemini via an API key, prompting it to summarize the text. API errors like `429` (Quota Reached) or `503` (Service Unavailable) happen when Google's servers are too busy.

ChromaDB & Vector Databases

Traditional databases search by exact keyword matches. A vector database (like ChromaDB) searches by *meaning*. When we store a document, we convert it into numbers (vectors). When a user asks a question, we convert the question into numbers and find the document chunks mathematically closest in meaning to the question.

SentenceTransformers (`all-MiniLM-L6-v2`)

This is the specific AI model that does the conversion from text to numbers (embeddings). We run this locally on the server to avoid paying API costs for embedding generation. It creates a 384-dimensional vector for every chunk of text.

Retrieval-Augmented Generation (RAG)

LLMs like Gemini don't inherently know about a user's private contract, nor do they perfectly memorize specific Indian Contract Law. RAG fixes this: 1. **User Question:** "What happens if they terminate early?" 2. **Search:** We search ChromaDB for clauses regarding "early termination" in the user's uploaded document. 3. **Retrieve:** ChromaDB returns the 3 most relevant paragraphs. 4. **Augment & Generate:** We send the question PLUS the retrieved paragraphs to Gemini, saying: "Answer the question USING ONLY this text." This guarantees accurate, grounded answers and prevents the AI from hallucinating.

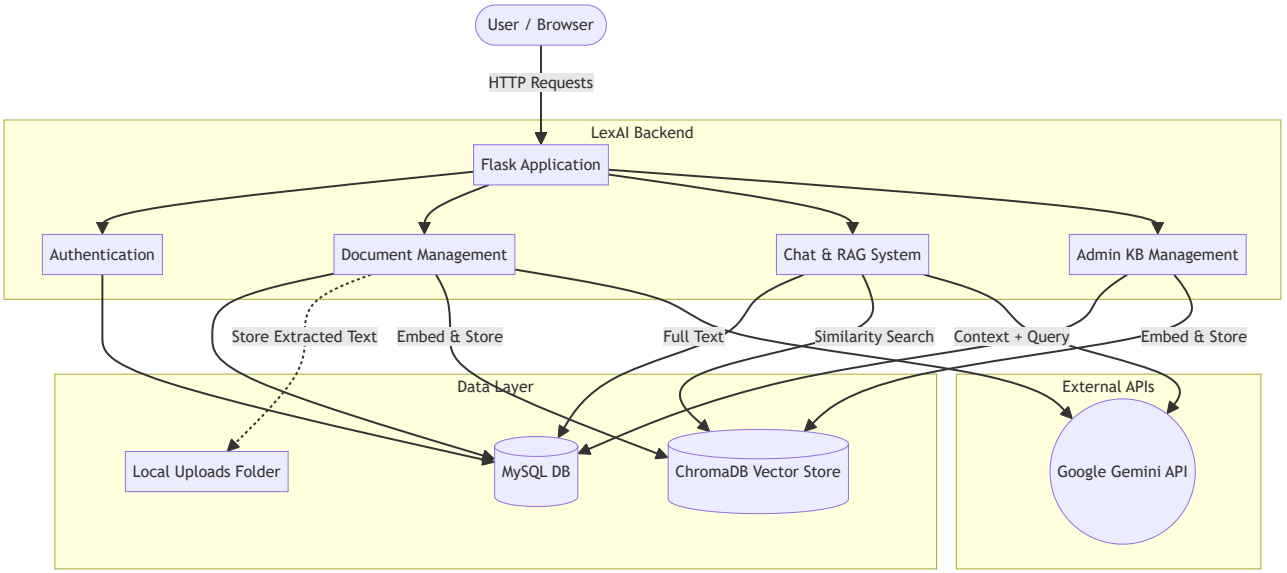
PyMuPDF & python-docx

These libraries open the binary `.pdf` and `.docx` files the user uploads and pull out raw text strings that our AI and embedding models can process.

marked.js + DOMPurify

Gemini returns responses formatted in Markdown (using `**bold**` and `### headings`). `marked.js` converts this to HTML so the browser can style it. However, directly inserting HTML into a webpage is dangerous (a hacker could inject malicious scripts). `DOMPurify` scrubs the HTML clean before we display it.

7. System Architecture



Explanation: - The user interacts with the Frontend (HTML/JS), sending requests to the Flask Backend. - The Backend parses documents and stores the physical files on the filesystem, while metadata goes to MySQL. - Text is chunked and embedded using local models, storing vectors in ChromaDB. - When querying, the Chat System searches ChromaDB for context, then sends the context and the user query to the Google Gemini API to generate a response.

8. Complete User Flow

Primary Application Journey

1. **Public Landing Page:** Visitor lands on `/` and views the SaaS-style marketing page.
2. **Authentication:** User navigates to `/register` or `/login` and authenticates.
3. **Dashboard:** User lands on `/dashboard`, seeing a summary of their uploaded documents.
4. **Document Upload:** User navigates to the AI Analysis page and uploads a `.pdf` or `.docx`.
5. **Processing:** The backend validates the file, extracts text, calculates word/page counts, and chunks/embeds the text into ChromaDB.

6. **Analysis Generation:** The backend sends the full text (if within limits) to Gemini to generate an Executive Summary, which is stored in MySQL.
7. **Document Chat:** The user opens the Chat UI (`/chat`), selects the uploaded document, and asks a specific question. The RAG system isolates the search to *only* that document's vector chunks and generates an answer.

General Legal Chat Journey

1. The user goes to `/chat` and selects "General Legal".
2. The user asks "What is a voidable contract?".
3. The RAG system searches ChromaDB filtering strictly for `source_type: knowledge_base` .
4. It retrieves matching legal concepts curated by the Admin.
5. Gemini uses those concepts to formulate a highly accurate legal answer. (If Gemini is down, the deterministic fallback parser kicks in to present the KB data cleanly).

9. Public Website vs Application

LexAI maintains a strict architectural and visual separation between the public-facing marketing site and the authenticated application dashboard.

PUBLIC WEBSITE - **Route:** `/` (handled by `main_routes.py`) - **Template:** `index.html` extending `public_base.html` - **Purpose:** Marketing, branding, feature highlights. - **UI:** No sidebars. Standalone SaaS-style navigation.

AUTHENTICATED APPLICATION - **Routes:** `/dashboard` , `/chat` , `/documents` , `/admin` , etc. - **Template:** Various, all extending `base.html` - **Purpose:** Actual functionality (uploading, chatting). - **UI:** Features a persistent left-hand navigation sidebar, user profile dropdowns, and protected route access.

10. Folder Structure

The following represents the ACTUAL structure of the LexAI project codebase:

```
text LexAI/ ├── .env.example # Template for required environment variables ├── app.py # Flask application factory and registration ├── config.py # Configuration classes (Dev, Test, Prod) ├── init_db.py # Script to initialize SQLAlchemy models ├── requirements.txt # Python dependencies ├── setup_db.py # Script to manually configure MySQL DBs │ ├── models/ │ └── database_models.py # SQLAlchemy definitions (User, Document, etc.) │ ├── routes/ │ ├── admin_routes.py # Admin dashboard and KB CRUD routes │ ├── auth_routes.py # Login, Register, Logout │ ├── chat_routes.py # API routes for General and Document chat │ ├── document_routes.py # Upload, Analysis, Document History │ └── main_routes.py # Public landing page, Dashboard, Profile │ ├── scripts/ │ └── seed_kb.py # Script to populate Knowledge Base with vectors │ ├── services/ │ ├── document_service.py # PDF/DOCX parsing, validation, stats │ ├── embedding_service.py # Chunking logic, SentenceTransformers │ └── rag_service.py # ChromaDB retrieval, Fallback logic, Gemini │ └── ai_service.py # Gemini analysis (Executive summary) │ ├── static/ │ ├── css/ │ │ └── style.css # Custom CSS, Dashboard layouts │ └── js/ │ ├── chatbot.js # AJAX chat, Markdown parsing, MathJax │ └── main.js # General UI scripts │ ├── templates/ # HTML Jinja2 templates │ ├── admin/ # Admin specific views │ ├── auth/ # Login/Register views │ ├── base.html # Dashboard layout skeleton │ ├── public_base.html # Public website skeleton │ ├── chatbot.html # Main Chat UI │ ├── index.html # Public landing page │ └── ... # Analysis, Documents, Dashboard views │ └── tests/ # Pytest automation suite ├── conftest.py # Fixtures and safe DB setup ├── test_advanced_rag.py # Tests for smart routing/summarization ├── test_ai_analysis.py # Tests for Gemini summarization ├── test_auth.py # Tests for login/registration ├── test_database.py # Tests for cascading deletes ├── test_document.py # Tests for upload validation ├── test_isolation.py # Tests ensuring Test DB doesn't hit Prod DB ├── test_kb_fallback.py # Tests for the deterministic fallback parser ├── test_phase5.py # Tests for admin routes and health checks ├── test_rag.py # Tests for ChromaDB retrieval └── test_security.py # Tests for unauthorized access and path traversal
```

11. File-By-File Responsibility

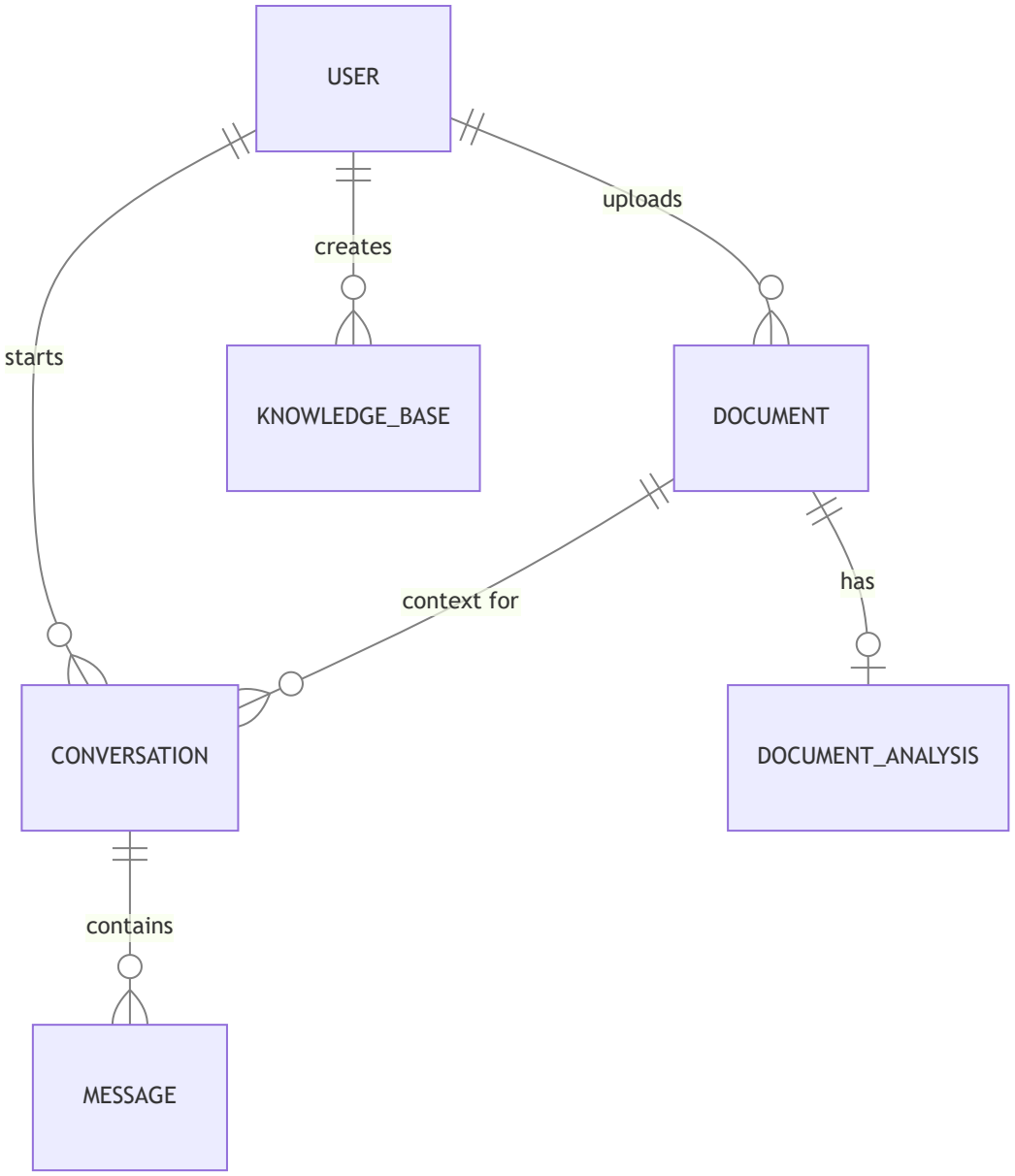
- `app.py` : The entry point. Implements the Application Factory pattern (`create_app()`), initializes database extensions, sets up login managers, and registers Blueprints.
- `config.py` : Holds `Config` , `DevelopmentConfig` , and `TestConfig` . Reads `.env` for database URIs and Gemini API keys.
- `models/database_models.py` : Defines the MySQL schema. Key tables: `User` , `Document` , `DocumentAnalysis` , `Conversation` , `Message` , `KnowledgeBase` .
- `routes/auth_routes.py` : Handles user creation, password hashing via Bcrypt, and Flask-Login session creation. Secures the `next` parameter to prevent Open Redirects.
- `routes/document_routes.py` : Handles multipart form uploads, interacts with `document_service` for parsing, stores records, triggers embeddings, and triggers AI summary generation.
- `routes/chat_routes.py` : Exposes `/api/chat` , receiving JSON payloads. Routes the query to `rag_service.answer_question()` and handles 429/500 HTTP status conversions.
- `routes/admin_routes.py` : Protected routes (`@admin_required`) allowing admins to add/delete Knowledge Base entries and view system metrics.
- `routes/main_routes.py` : Handles the public `/` landing page, the protected `/dashboard` , user `/profile` , and the public `/knowledge-base` directory.
- `services/document_service.py` : Validates allowed extensions, sizes, and file content. Uses `fitz` (PyMuPDF) and `docx` to extract text and calculate page counts safely.
- `services/embedding_service.py` : Manages the local `SentenceTransformer` model. Includes the text chunking algorithm (overlapping window) to prepare documents for vectorization.
- `services/rag_service.py` : The "brain" of the application. Queries ChromaDB based on `document_id` or `knowledge_base` filters. Sends retrieved context to Gemini via the `genai` SDK. Implements the robust, deterministic fallback parser when Gemini is down.
- `ai_service.py` : Contains `analyze_legal_document()` , the dedicated function to prompt Gemini for the structured JSON Executive Summary.
- `scripts/seed_kb.py` : An admin script that generates synthetic legal knowledge base articles via Gemini, embeds them, and stores them in MySQL and ChromaDB to initialize the platform.

12. Database Architecture

Database Engine: MySQL
Production Database Name: `lexai_db`
Test Database Name: `LexAI_test` (SQLite)

Table Structures & Relationships

1. `users`
2. **Purpose:** Stores registered accounts and admin roles.
3. **Columns:** `id` (PK), `email`, `password_hash`, `name`, `role` ('user' or 'admin').
4. **Relationships:** Owns Documents, Conversations, and created KnowledgeBase articles.
5. `documents`
6. **Purpose:** Metadata for uploaded legal files.
7. **Columns:** `id` (PK), `user_id` (FK), `original_filename`, `file_path`, `file_size`, `word_count`, `page_count`.
8. **Relationships:** Owned by User. One-to-One with DocumentAnalysis. Deleted if User is deleted.
9. `document_analysis`
10. **Purpose:** Caches the Gemini-generated analysis to prevent re-querying the API.
11. **Columns:** `id` (PK), `document_id` (FK), `executive_summary`, `structured_data` (JSON).
12. `knowledge_base`
13. **Purpose:** Stores curated legal articles for General Legal Chat.
14. **Columns:** `id` (PK), `title`, `category`, `created_by` (FK to User).
15. `conversations`
16. **Purpose:** Groups a series of chat messages.
17. **Columns:** `id` (PK), `user_id` (FK), `document_id` (FK, Nullable - null means it's a General Chat).
18. `messages`
19. **Purpose:** Individual chat bubbles.
20. **Columns:** `id` (PK), `conversation_id` (FK), `sender` ('user' or 'ai'), `content` (TEXT).



13. Document Processing Pipeline

When a user clicks "Upload Document": 1. **Validation:** `document_routes` checks if the file is `.pdf` or `.docx` and under the 16MB limit. Security checks ensure it's not a disguised script or malware. 2. **File Storage:** The file is saved securely to `uploads/documents/<user_id>/` . 3. **Text Extraction:** `document_service` processes the binary file, stripping layout and extracting raw unicode text. 4. **Statistics:** Word counts and page counts are calculated and saved to the `documents` table in MySQL. 5. **Chunking:** `embedding_service` slices the massive string into smaller, overlapping chunks (e.g., 500 words with 50-word overlaps) so context isn't lost across chunk boundaries. 6. **Embedding & Indexing:**

The `SentenceTransformer` runs on each chunk, generating vectors that are stored in the local `chroma_db/` folder, tagged with `{"document_id": ID, "user_id": ID}` metadata.

14. AI Legal Analysis Pipeline

After a document is processed, LexAI automatically generates a structured analysis.

The Flow: 1. The backend passes the extracted text to `ai_service.analyze_legal_document()`. 2. A strict system prompt is attached, forcing Gemini to reply in a specific JSON structure. 3. Gemini processes the document and returns the JSON. 4. The backend parses the JSON, creating an Executive Summary and extracting Key Clauses. 5. The data is saved into the `document_analysis` MySQL table. 6. The `analysis.html` template renders this data dynamically in the browser.

Error Handling / Resilience: - If Gemini returns `503 Unavailable` or `429 Quota Exceeded`, the backend catches the error. - The analysis is marked as *failed*, and the user is shown a friendly error message ("Model experiencing high demand"). - A "Retry Analysis" button appears on the UI, allowing the user to trigger the analysis later without re-uploading the file. - If Gemini returns malformed text instead of JSON, the backend attempts to aggressively clean the string or safely defaults to displaying a raw summary.

15. RAG System Explained

RAG (Retrieval-Augmented Generation) is what prevents LexAI from hallucinating and allows it to securely answer questions about private documents.

The Concept of a Chunk: Language models have token limits. We cannot feed a 500-page document into Gemini for every chat question. Instead, we break the document into "chunks".

The Embedder: `all-MiniLM-L6-v2` turns chunks into vectors. Vectors are essentially mathematical coordinates. Sentences with similar meanings get coordinates close to each other.

ChromaDB Retrieval: When a user asks "Who is liable?", that question is embedded into a vector. ChromaDB calculates the mathematical distance between the question vector and all document chunk vectors, returning the top 3 closest chunks.

Document Chat Mode

- **Filtering:** ChromaDB query includes `where={"document_id": user_selected_id}`.
- **Isolation:** Because the database validates that `document.user_id == current_user.id`, User A can *never* query User B's document chunks.
- **Generation:** Gemini receives a prompt: "Use the following extracted document chunks to answer...".

General Legal Chat Mode

- **Filtering:** ChromaDB query includes `where={"source_type": "knowledge_base"}`.
- **Generation:** Gemini acts as a legal assistant, grounded *only* by the approved knowledge base articles.

16. Knowledge Base

The Knowledge Base (KB) is a curated repository of general legal concepts (e.g., "Breach of Contract", "NDAs"). It prevents LexAI from giving unauthorized or wild legal advice by restricting general chat answers to approved topics.

Management & Storage: - Admins can view, add, and delete articles via the `/admin` interface. - Title and Category metadata is stored in MySQL. - The actual article content is embedded and stored in ChromaDB using `source_type: knowledge_base`.

KB Seeding (`seed_kb.py`): To bootstrap the platform, `scripts/seed_kb.py` iterates through a list of fundamental legal topics. It calls Gemini to generate comprehensive articles for each topic, embeds them, and inserts them into both MySQL and ChromaDB. Due to Gemini's free-tier rate limits (15 requests per minute), the script includes automatic 60-second sleep cycles.

The Deterministic KB Fallback Mechanism: If Gemini goes down (HTTP 503 or 429) during a General Legal Chat, the RAG system intercepts the failure. Because ChromaDB successfully retrieved the relevant KB chunks *before* the Gemini failure, a custom python parser (`construct_fallback_response`) kicks in. It deterministically strips Markdown metadata out of the database chunk, isolates the Direct Answer, Key Information, and Remedies based on the user's query keywords, and returns a perfectly formatted Markdown response—ensuring the user still gets a reliable answer without needing the AI generation step!

17. Chat System

The Chat System relies on AJAX requests to provide a seamless, non-reloading interface.

1. **Frontend:** `chatbot.js` intercepts the form submission. It appends a temporary "Loading" bubble and sends a `POST /api/chat` request containing the `message`, `conversation_id`, and `document_id`.
2. **Backend:** The route validates ownership, queries the RAG service, and gets an answer.
3. **Storage:** It saves the user's message and the AI's response to the `messages` MySQL table, associated with a `conversation_id`.
4. **Return:** It returns JSON containing the `answer` and the `sources`.
5. **Rendering:** `chatbot.js` passes the answer through `marked.js` (for Markdown formatting), `DOMPurify` (for security), and finally calls `MathJax` to format any embedded mathematical equations (common in contract logic explanations).

18. Authentication & Security

LexAI employs robust security measures: - **Password Security:** Plaintext passwords are never stored. `Flask-Bcrypt` hashes passwords with salt. - **Session Management:** `Flask-Login` handles HttpOnly session cookies. - **Role-Based Access Control (RBAC):** Admin routes are protected by an `@admin_required` decorator. If a normal user tries to access `/admin`, they are redirected. - **Data Isolation:** Every single database query for documents or conversations includes `.filter_by(user_id=current_user.id)`. Path traversal attacks (e.g., trying to read `../../etc/passwd`) are blocked using `werkzeug.utils.secure_filename`. - **Open Redirect Protection:** The login system validates that the `next` URL is a relative path (local) and not an external malicious site. - **Cross-Site Scripting (XSS):** `DOMPurify` aggressively scrubs all AI-generated output on the frontend to ensure no `<script>` tags are executed.

19. Error Handling

- **HTTP 404 (Not Found):** Handled gracefully with a custom `404.html` page for browsers, and a standard JSON response for API endpoints.

- **HTTP 413 (Payload Too Large):** Triggers if a user uploads a file > 16MB. Returns a friendly flash message.
- **HTTP 429 (Too Many Requests):** Used specifically when Gemini hits its rate limit. The chatbot frontend catches this and displays the deterministic fallback or a friendly retry message.
- **HTTP 500/503 (Server/Service Unavailable):** Caught during RAG processing. The user is informed that the model is experiencing high demand.

20. Testing

LexAI includes a rigorous, production-grade Pytest automation suite designed to protect the codebase against regressions.

Framework: `pytest`
Latest Test Result: 86/86 Tests Passed (0 Failures)

Isolation Strategy: The tests **never** touch production data. `tests/conftest.py` actively forces the application to use an in-memory or SQLite database (`LexAI_test`) and a separate ephemeral ChromaDB directory (`chroma_db_test/`). The suite includes fail-safes that immediately abort testing if they detect the production database URI.

Key Test Categories: - `test_auth.py` : Verifies login logic, short passwords, duplicate emails. - `test_security.py` : Proves User B cannot access User A's documents, chats, or RAG vectors. Proves path traversal and malicious executable uploads are blocked. - `test_rag.py` : Verifies vector chunking lengths, overlap logic, and empty-file handling. - `test_kb_fallback.py` : Mocks `google.genai.Client` to simulate network failures, proving the deterministic fallback parser generates clean, truncated Markdown without raw database separators. - `test_database.py` : Ensures cascading deletes work (deleting a user successfully deletes their documents and chat history).

21. Project Development Phases

PHASE 1: Foundation (Implemented) - Set up Flask, MySQL, and SQLAlchemy. - Implemented User Authentication, Password Hashing, and Dashboard UI.

PHASE 2: Document Management (Implemented) - Built file upload system with security validations. - Integrated PyMuPDF and python-docx for text extraction and word counting.

PHASE 3: AI Legal Analysis (Implemented) - Connected Google Gemini API. - Implemented structured JSON prompts for Executive Summaries. - Built the Analysis UI to display extracted clauses.

PHASE 4: RAG & Chat Systems (Implemented) - Set up local SentenceTransformers and ChromaDB. - Implemented text chunking. - Built the Chatbot frontend and connected it to the RAG backend, ensuring strict user-document isolation.

PHASE 5: Knowledge Base & Admin (Implemented) - Developed the Admin Dashboard and KB CRUD interface. - Created `seed_kb.py` to auto-generate the legal knowledge corpus. - Separated General Legal Chat from Document Chat.

PHASE 6: Optimization & Stability (Implemented) - Fixed Open Redirect vulnerabilities. - Extracted main routes into Blueprints for cleaner architecture. - Replaced deprecated SQLAlchemy APIs to resolve warnings. - Built the deterministic 503 KB Fallback mechanism and frontend MathJax rendering.

22. Current Project Status

WHERE WE ARE NOW The project is fundamentally complete for its core requirements. - **Completed & Working:** Authentication, Uploads, RAG, Chat, AI Summarization, Admin Panel, Knowledge Base, Automated Testing. - **Stability:** The system passes 86/86 tests. All critical bugs, routing issues, and AI fallback parsing issues have been completely resolved. - **Warnings:** Only 3 minor warnings remain during testing, which originate entirely from third-party upstream dependencies (e.g., a deprecation inside the `chromadb` telemetry module), not the LexAI codebase itself. - **Pending / Future Work:** Cloud deployment, OCR, multilingual support.

23. Gemini API / Quota

LexAI heavily relies on the Google Gemini API (`GEMINI_API_KEY`). - The key is configured in the `.env` file. It acts as a secret password and **must never be committed to GitHub**. - The `AI_MODEL` variable defines which Gemini version to use (e.g., `gemini-2.5-flash`). - **Quota / 429 Error:** The free tier of Gemini limits users to 15 requests per minute. If you click too rapidly or run the seed script too fast, Google blocks the request. LexAI handles this gracefully. - **Service Unavailable / 503 Error:** Sometimes Google's servers are overloaded. This is external to LexAI.

24. Environment Variables

The project requires a `.env` file in the root directory. Below is the reference list (derived from `.env.example`):

- `FLASK_APP=app.py` : Tells Flask where the app starts.
- `FLASK_ENV=development` : Turns on development mode (do not use in production).
- `FLASK_DEBUG=1` : Enables the interactive debugger.
- `SECRET_KEY=YOUR_SECRET_KEY` : Secures user session cookies. Make this random.
- `DB_HOST=localhost` : The MySQL server address.
- `DB_USER=root` : Your MySQL username.
- `DB_PASSWORD=YOUR_DB_PASSWORD` : Your MySQL password.
- `DB_NAME=lexai_db` : The name of the MySQL database.
- `GEMINI_API_KEY=YOUR_GEMINI_API_KEY` : Your personal Google AI API key.
- `AI_MODEL=gemini-2.5-flash` : The targeted LLM.

(Note: Real keys are strictly excluded from this documentation.)

25. How to Setup Project Locally

For new teammates joining development on Windows:

1. **Clone the repository.**
2. **Create a virtual environment:** `python -m venv venv`
3. **Activate it:** `venv\Scripts\activate` (or `. venv/bin/activate` on Mac/Linux)
4. **Install dependencies:** `pip install -r requirements.txt`
5. **Setup MySQL:** Ensure MySQL Server is running locally.
6. **Configure `.env` :** Copy `.env.example` to `.env` and fill in your DB credentials and Gemini key.
7. **Create Databases:** Run `python setup_db.py` to create `lexai_db` and `LexAI_test` .
8. **Initialize Tables:** Run `python init_db.py` to create the schema and an initial admin account.

9. **Seed Knowledge Base (Optional but recommended):** Run `python scripts/seed_kb.py` to populate the AI knowledge base. *(Note: Takes a few minutes due to API rate limits).*

26. How to Run the Project

Start the web server: `powershell venv\Scripts⍰ctivate python app.py` Then navigate to `http://127.0.0.1:5000` in your browser.

Run the Automated Tests: `powershell venv\Scripts⍰ctivate python -m pytest -v`

27. How to Add a New Knowledge Base Article

There are two ways to add articles to the KB: 1. **Via UI:** Login as an Admin, click "Admin Dashboard" in the sidebar, and use the "Add Knowledge Base Entry" form. This saves the metadata to MySQL and automatically embeds the content into ChromaDB. 2. **Via Script:** Add a tuple to the `kb_topics` list in `scripts/seed_kb.py` and run the script. It will call Gemini to write the article, embed it, and store it.

28. How to Debug Common Problems

Problem	Possible Cause	How to Fix
Gemini 429 Error	Rate limit exceeded (15 RPM).	Wait 60 seconds and try again.
Database Connection Failure	MySQL is not running or wrong `.env` credentials.	Open Services, start MySQL. Check `DB_PASSWORD`.
Empty AI Analysis	Gemini returned malformed JSON or errored out.	Check the server console logs. Click "Retry Analysis" in UI.
Missing Module Error	Virtual environment not active.	Run `venv\Scripts\activate` before `python app.py`.
Routing `BuildError`	Missing Blueprint prefix in `url_for`.	Ensure `url_for('index')` is written as `url_for('main.index')`.

> **CRITICAL DATABASE WARNING:**
> Never run `db.drop_all()` or delete the `chroma_db/` folder in production unless you intend to completely destroy all user data, documents, and embeddings. Use migrations for schema updates.

29. Development Rules for Teammates

1. **Never commit `.env` :** Keep your API keys and passwords safe.
2. **Never run destructive commands:** Do not wipe the database to fix a small issue.
3. **Never point tests to production:** Ensure `conftest.py` is intact. Tests must ALWAYS run against `LexAI_test`.
4. **Run tests before pushing:** Run `python -m pytest -v`. Do not push if it's below 86/86.
5. **Preserve Document Isolation:** When writing database queries, ALWAYS include `.filter_by(user_id=current_user.id)` to prevent users from seeing other people's data.
6. **Keep Frontend/Backend separate:** Put business logic in `services/`, route definitions in `routes/`, and UI in `templates/`.

30. Project Limitations

- **Gemini API Limits:** Heavy usage by multiple simultaneous users will exhaust the free-tier API limits quickly.
- **Local Storage:** Uploaded files and ChromaDB vectors are currently stored locally on the server disk. This makes load-balancing across multiple servers difficult without moving to cloud object storage.
- **No Replacement for Professionals:** The AI can hallucinate or misinterpret complex jurisdictional nuances. LexAI explicitly warns users that it is an assistant, not a lawyer.
- **OCR Missing:** It cannot read scanned image PDFs, only text-based PDFs.

31. Future Scope

(Not currently implemented) - **Cloud Deployment:** Migrating the database to Google Cloud SQL and storage to Firebase. - **Multilingual Support:** Utilizing Gemini to translate legal advice into local regional languages. - **Advanced Document Comparison:** Uploading two versions of a contract and having the AI highlight exact clause differences. - **Scalable Vector Database:** Replacing local ChromaDB with Pinecone or managed Vertex AI Vector Search.

32. Team Handover Section

IF YOU ARE A NEW DEVELOPER ON LEXAI, START HERE:

1. Read the `README.md` for a high-level summary.
2. Read `config.py` to understand how the environment is set up.
3. Read `models/database_models.py` to understand the data relationships.
4. Dive into `services/rag_service.py` —this is the most complex and important file in the system.

Where to look to modify specific features: - **Changing Login/Registration:** Go to `routes/auth_routes.py` and `templates/auth/` . - **Changing AI Prompts:** Go to `ai_service.py` (for Analysis) and `services/rag_service.py` (for Chat). - **Tweaking Chunk Sizes:** Go to `services/embedding_service.py` . - **Fixing Chat UI Bugs:** Go to `static/js/chatbot.js` and `templates/chatbot.html` . - **Adding New Pages:** Register them in `routes/main_routes.py` and link them in `templates/base.html` .

33. Glossary

- **LLM (Large Language Model):** An AI trained on massive amounts of text, capable of generating human-like responses (e.g., Gemini).
- **ORM (Object-Relational Mapper):** A tool (like SQLAlchemy) that lets developers query a database using Python instead of raw SQL.
- **Embedding:** A mathematical representation (vector array) of text, allowing computers to measure semantic similarity.
- **Chunk:** A small piece of a larger document, broken down so it fits within an AI's token limit.
- **JSON:** A standard text format for structuring data, used for API communications and structured AI outputs.
- **RBAC (Role-Based Access Control):** Restricting system access based on the user's role (admin vs. user).
- **XSS (Cross-Site Scripting):** A vulnerability where attackers inject malicious scripts into web pages. Prevented in LexAI using `DOMPurify` .
- **HTTP 429:** A status code indicating "Too Many Requests" (Rate Limit hit).
- **HTTP 503:** A status code indicating the service is temporarily overloaded or unavailable.